Weather Fusion Dashboard

ระบบรวบรวมสภาพอากาศหลายแหล่งด้วย Apache Airflow + PostgreSQL + Flask Dashboard

> **วิชา:** 02739452 **ลำดับที่:** 02 **รหัสนิสิต:** 6621650574
> **GitHub repo:** https://github.com/Namsai-ns/02739452-02-6621650574
> **ชื่อ repo:** 02739452-02-6621650574 (public)

1. หัวข้อและวัตถุประสงค์

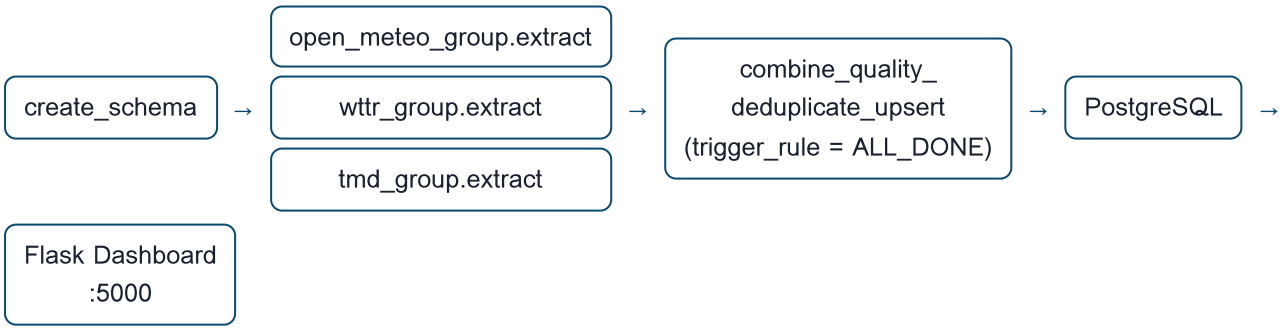
สร้าง data pipeline แบบ ETL ที่ดึงข้อมูลสภาพอากาศจากหลาย API ปรับให้อยู่ใน schema เดียวกัน ตรวจคุณภาพข้อมูล บันทึก (UPSERT) ลง PostgreSQL แล้วแสดงผลผ่านเว็บ Dashboard โดย DAG ตั้งเวลาทำงานทุก 6 ชั่วโมง (`0 */6 * * *`)

2. แหล่งข้อมูล

แหล่ง	รายละเอียด
Open-Meteo Forecast API	ไม่ต้องใช้ API key; ดึง current weather ของ 77 จังหวัดด้วยพิกัดตัวเมืองของแต่ละจังหวัด (แหล่งหลักของระบบ)
wttr.in JSON API	ไม่ต้องใช้ API key; ดึง 5 พื้นที่ (กรุงเทพฯ, เชียงใหม่, ภูเก็ต, ขอนแก่น, นครราชสีมา)
กรมอุตุนิยมวิทยา (TMD) WeatherToday API	endpoint สาธารณะ ตั้งค่าผ่าน `TMD_API_URL` ใน `.env`; มี parser แบบยืดหยุ่นเพราะ schema อาจเปลี่ยน

3. สถาปัตยกรรมและขั้นตอนของ DAG

DAG `weather_multi_source_pipeline` ทำงานตามลำดับด้านล่าง



- **ดึงข้อมูลขนาน:** TaskGroup แยก 3 แหล่ง; แต่ละ task จับ error ของตัวเองและส่งสถานะกลับ ถ้าแหล่งหนึ่งล่ม แหล่งอื่นยังทำงานต่อได้
- **Normalize:** source, location, observed_at, temperature_c, humidity_pct, rainfall_mm, latitude, longitude, weather_description
- **Quality check:** ต้องมี location และ observed_at; temperature −90..65 °C; humidity 0..100 %; rainfall 0..2000 mm; ห้ามวัดค่าเป็น null ทั้งหมด — แถวที่ไม่ผ่านเก็บใน `weather_rejected` พร้อมเหตุผล
- **UPSERT:** unique key `(source, location, observed_at)` รัน DAG ซ้ำแล้วไม่เกิดแถวซ้ำ

4. ฐานข้อมูล (PostgreSQL 16)

ตาราง	หน้าที่
weather_observations	ข้อมูลอากาศที่ผ่านการตรวจ (มี unique constraint และ index ตามเวลา/สถานที่)
weather_rejected	ข้อมูลที่ไม่ผ่าน quality check (raw_record เป็น JSONB + reject_reason)
pipeline_runs	สถานะแต่ละแหล่งต่อ DAG run (status, records_received, records_loaded, error_message)

5. เว็บ Dashboard (โฟลเดอร์ web/)

Flask app พอร์ต 5000 แสดง KPI, กราฟอุณหภูมิตามเวลา, ตารางเปรียบเทียบอุณหภูมิรายจังหวัด, ตัวกรอง และ Pipeline Status ของแต่ละแหล่ง ผ่าน API `/api/dashboard`, `/api/filters`, `/api/pipeline`

6. โครงสร้าง repo

```
dags/weather_pipeline.py     Airflow DAG
web/app.py, web/templates/   Flask dashboard
web/requirements.txt         dependencies ของ web (ใช้ใน Docker)
requirements.txt             dependencies รวม (รันนอก Docker)
docker-compose.yml           postgres, airflow-init/webserver/scheduler, dashboard
Dockerfile.airflow / Dockerfile.dashboard
sql/check.sql                SQL ตรวจผลลัพธ์
.env.example                 ตัวอย่างค่าตั้งต้น (ไม่ commit .env จริง; .env อยู่ใน .gitignore)
README.md                    หัวข้อ แหล่งข้อมูล วิธีรัน
```

7. วิธีรัน

```
git clone https://github.com/Namsai-ns/02739452-02-6621650574.git
cd 02739452-02-6621650574
cp .env.example .env
docker compose up --build airflow-init
docker compose up --build -d
```

- Airflow: `http://localhost:8080` (ผู้ใช้ `airflow` / รหัสผ่าน `airflow` — ค่า default สำหรับ local เท่านั้น)
- Dashboard: `http://localhost:5000`
- ใน Airflow เปิด DAG `weather_multi_source_pipeline` แล้วกด Trigger DAG จากนั้นรีเฟรช Dashboard
- ตรวจข้อมูลด้วย SQL: `sql/check.sql` (PostgreSQL เปิดที่พอร์ต 5433 ของเครื่อง)

8. วิธีทดสอบ

- **รันซ้ำ:** Trigger DAG สองครั้ง จำนวนแถวของ key เดิมต้องไม่เพิ่ม
- **จำลองแหล่งล่ม:** ตั้ง `ENABLE_WTTR=false` หรือ `ENABLE_TMD=false` ใน `.env` แล้ว `docker compose up -d` ใหม่ แล้วรัน DAG — แหล่งอื่นต้องโหลดต่อได้ และหน้า Pipeline Status แสดงสถานะของแต่ละแหล่ง
- **ข้อมูลไม่ผ่าน:** ตรวจตาราง `weather_rejected`

9. ข้อจำกัด

- ข้อมูลที่แสดงเป็นเฉพาะจังหวัดที่ API ส่งมาได้จริงในรอบนั้น (log ของ `open_meteo_group.extract` แสดง `province coverage: x/77`)
- wttr.in ครอบคลุม 5 พื้นที่ และ TMD ขึ้นกับสิทธิ์/schema ของ endpoint ณ เวลานั้น จึงไม่ควรตีความว่าทั้งสามแหล่งครบทุกจังหวัด; ไม่ควรเปรียบเทียบข้อมูลที่เวลา/สถานีต่างกันโดยตรง (เก็บ `source` ทุกแถว)